thunk — Product Requirements Document (PRD)

Product: thunk — an offline systems course for justice-impacted learners **Author:** Penn Porterfield · **Status:** Draft (Phase 1 built) · **Date:** 2026-07 **Related:** `DESIGN-SPEC.md` (architecture) · `superpowers/plans/2026-07-06-thunk-phase1.md` (build plan) · `thunk-risk-assessment.html` (risks) · `PROPOSAL.md` (the pitch)

---

1. SUMMARY

thunk is a single, self-contained, fully offline Rust program that teaches the low level of a computer from true zero, ending with DOOM booting on a display the learner drove from the bare metal up. The entire hardware stack is simulated in software, so it runs on a locked-down machine with no network and no hardware and clears facility review. The same course drives real hardware on the open build. It is built around the reentry window and ends in tangible, public artifacts — a driver and real open-source contributions.

2. PROBLEM

- Coding programs for justice-impacted people teach web development almost exclusively (The Last Mile, Persevere, Justice Through Code). Entry-level web is saturated and shrinking (web median $90,930 vs $133,080 software dev; junior software employment down ~20% since 2022).
- No prison or reentry program teaches systems/embedded/kernel — a verified white space.
- The reason it stayed empty: low-level work needs hardware, and hardware does not clear facility review.
- Reentry outcomes hinge on demonstrable, verifiable skill: 76.6% are rearrested within five years; formerly-incarcerated unemployment is ~27%; a record halves callbacks. Correctional education cuts the odds of returning to prison by 43%.

3. GOALS AND NON-GOALS

Goals - Teach real systems fundamentals to true-zero beginners, self-paced, with no instructor required. - Run entirely offline on a single locked-down machine and clear facility review. - End every learner at a visible payoff (DOOM on a simulated panel) with zero hardware. - Produce tangible, resume-ready artifacts, culminating in real open-source contributions. - Serve both inside-the-wall and reentry audiences from one curriculum.

Non-goals (for now) - Not a web-development course; not a competitor to existing programs, a complement. - Not a graded, cohort-locked, instructor-dependent program. - Not dependent on any network service, cloud, account, or telemetry. - Not shipping the open-source/first-patch track on the inside build (it needs the internet).

4. TARGET USERS

- **Inside learner** — currently incarcerated, air-gapped machine, often true-zero. Primary.
- **Reentry learner** — recently released; wants a portfolio and a first open-source contribution.
- **Experienced systems learner** — arrives with real background; must be able to test out, not sit through the on-ramp.
- **Facilitator / mentor** (optional) — runs a cohort; needs pacing, answer keys, progress view.

5. FUNCTIONAL REQUIREMENTS (USER STORIES)

Learning experience - FR-1: A learner can read a lesson and move through a module at their own pace. - FR-2: A learner answers checks (multiple-choice and short-answer) and gets immediate feedback. - FR-3: The course is mastery-gated: passing a module's checks unlocks the next (competency gates). - FR-4: A placement diagnostic lets an experienced learner test out of modules they already know. - FR-5: Progress (lessons read, checks passed, module mastery) is tracked and visible. - FR-6: The experience cannot get stuck — help is always available, no dead ends, no failing grade.

Modes (one content source) - FR-7: A command-line mode (read / check / progress / sim). - FR-8: A terminal classroom (TUI): reader, interactive checks, panel view, help. - FR-9: A local web GUI (offline; `serve` binds 127.0.0.1 only) — richest visual mode. *(Phase 2.)*

Simulator (the keystone) - FR-10: A deterministic, pure-Rust model of an SPI bus that records a trace. - FR-11: A display-panel framebuffer the learner can drive; a boot splash renders over the bus. - FR-12: The finale boots DOOM on the simulated panel. *(Phase 2.)* - FR-13: The same interface (`SpiBus` trait) swaps to a real Saleae + panel on the open build.

Curriculum ladder - FR-14: M0 Power On → M1 The Kernel → M2 Rust for the Metal → M3 The Bus → M4 The Panel → M5 DOOM → M6 Intro to Open Source → M7 First Patch.

Open-source track (open build) - FR-15: M6 teaches what open source is: licenses, version control in the open, project/community norms, reading a diff, the culture of merit. - FR-16: M7 guides a first real contribution (docs/staging first, kernel patch as the ceiling), as a small peer community plugged into kernelnewbies / LKMP / Outreachy, with mentor pre-review.

Deployment - FR-17: Two build profiles chosen at build time: an air-gapped inside build (no hardware/network code present) and an open build (adds the hardware seam and the open-source track). - FR-18: An optional facilitator kit (pacing, answer keys, cohort/progress view). *(Phase 3.)*

6. NON-FUNCTIONAL REQUIREMENTS

- NFR-1 (Deployability): a single static-musl binary and/or a static web bundle; no installer, no admin rights.
- NFR-2 (Offline): no network or sockets at runtime; no auto-fetched resources; assets embedded at compile time.

- NFR-3 (Review-safety): no cryptography; no exploitation vocabulary (enforced by a `vocab-lint` CI gate); framing is engineering, not hacking. Inside build compiles with no hardware/network code present at all.
- NFR-4 (Privacy): minimal data; learner state is local only; no accounts, tracking, or telemetry.
- NFR-5 (Accessibility): true-zero entry, self-paced, plain language. *Known gap: screen-reader support and non-English localization are not yet addressed.*
- NFR-6 (Reproducible & open): auditable builds; source public under MIT/Apache-2.0.
- NFR-7 (Testable): domain and simulator logic pure and unit-tested; CI runs fmt, clippy (deny warnings), tests, and vocab-lint.

7. SCOPE AND PHASING

- **Phase 1 (built + passing tests):** workspace; Module 1 authored end to end (5 lessons, 15 checks); content pipeline into CLI + TUI; first simulator slice (bus + panel + boot splash); review-safety gate. Demoable at the Aug 2026 Next Chapter demo.
- **Phase 2:** full SPI/panel/DOOM simulator + trace view + sim-or-real seam; the web GUI; full M0–M5 curriculum; competency gates + placement diagnostic.
- **Phase 3:** the M6–M7 open-source track (own sub-spec); facilitator kit; hardened build profiles; Saleae/ partner outreach; first outside cohort; first facility conversation.

8. SUCCESS METRICS

- Learners who complete Module 1; checks mastered per learner.
- Learners who reach the DOOM finale (simulated).
- First real open-source contributions merged (reentry cohort).
- Placement diagnostic accuracy (experienced learners correctly skipped ahead).
- Facility review outcomes (builds approved to run inside).

9. CONSTRAINTS AND ASSUMPTIONS

- Solo builder; Aug 2026 demo deadline; ~25 hr/week alongside the kernel capstone.
- No funding secured (free + open source; Saleae is intended outreach, not a program).
- Facility review is out of the team's control and is the top residual risk.
- Assumes learners have access to a stock (often locked-down) computer, not hardware.

10. RISKS

See `thunk-risk-assessment.html` (D/V/F register with L×C scoring). Top residual: facility review.

11. OPEN QUESTIONS

- Workspace shape for the simulator (new `thunk-sim` crates vs. feature-gated).
- Whether the M0 on-ramp needs a runnable code sandbox given the no-arbitrary-execution constraint.
- How much of M6–M7 (patch etiquette) can be practiced offline inside vs. only live on the open build.
- Final name (thunk is a working title).

12. CURRENT STATUS

Phase 1 is built and green: four-plus-one crate workspace (`thunk-core/content/sim/tui/cli`), Module 1 authored, CLI + TUI working, first simulator slice rendering a boot splash, 24 tests passing, offline static build, dual license, and the vocab-lint gate. Local git only; not yet pushed.